

WEEK 3 : LECTURE 3 - CLASS MODELS & DIAGRAMS

Bachelor's Degree in Information Technology
BITSD4111, SOFTWARE DESIGN

Trimester: 06

From **March 2024** to **June 2024**

OBJECTIVES

- Class models.
- Finding classes and associations.
- Basics of UML Class Diagrams.
- ATM Case Study: initial class model

CLASS MODELS.

- There are 3 types of models in the object oriented modeling and design are: Class Model, State Model, and Interaction Model
- For now we will concentrate on the Class Model
- Class Model: The class model shows all the classes present in the system.
- The class model shows the attributes and the behavior associated with the objects.

CLASS MODELS.

- The class diagram is used to show the class model.
 - The class diagram shows the class name followed by the attributes followed by the functions or the methods that are associated with the object of the class.
 - The goal in constructing class model is to capture those concepts from the real world that are important to an application.

CLASS MODELS.

- Class models are part of requirement analysis
- Once requirements have been collected and captured, they need to be analyzed
- The purpose of requirements analysis is to produce an analysis, or logical, model of the planned application
- The analysis model can be seen as a necessary and important step between the requirements model, and the design model

FINDING CLASSES AND ASSOCIATIONS.

- Analysis is an attempt to build a model that describes the application domain -- developers do this
- It takes place after (or during) requirements specification
- The analysis model will typically consist of all three types of models:
 - Functional model (denoted with use cases)
 - Analysis object (class) model
 - Dynamic (Interaction) model

FINDING CLASSES AND ASSOCIATIONS.

- At this level, note that we are still looking at the application domain, not yet system design
- However, many things discovered in analysis could translate closely into the system design
- The goal is to completely understand the application domain (the problem at hand, any constraints that must be adhered to, etc.)
 - New insights gained during analysis might cause requirements to be updated.

FINDING CLASSES AND ASSOCIATIONS.

- Analysis activities include:
 - Identifying objects (often from use cases for a start)
 - Identifying associations between objects
 - Identifying general attributes and responsibilities of objects
 - Modeling interactions between objects
 - Modeling how individual objects change state -- helps identify operations
 - Checking the model against requirements, making adjustments, iterating through the process more than once

FINDING CLASSES AND ASSOCIATIONS

- When we think of objects in code, we want to represent in the real world, although not always true.
- Here are some categories of objects to look for:
 - Entity objects : represent persistent information tracked by a system. Closest to "real world" objects.
 - Boundary objects -- these represent interactions between user and system. (For instance, a button, a form, a display)
 - Control objects -- usually set up to manage a given usage of the system. Often represent the control of some activity performed by a system

FINDING CLASSES AND ASSOCIATIONS

- UML diagrams can include a label known as a stereotype, above the class name in a class diagram. This would be placed inside << >> marks, like this:
 - <<entity>>
 - <<boundary>>
 - <<control>>
- There are some different popular techniques for identifying objects. Two traditional and popular ones that we will discuss are:
 - natural language analysis (i.e. parts of speech)
 - CRC cards

FINDING CLASSES AND ASSOCIATIONS

Using Natural Language Analysis

- It is not perfect, but coupled with other techniques, it's a good start
- This can be done from a general problem description, or better, from a use case or scenario
- Map parts of speech to object model components.
 - nouns usually map to classes, objects, or attributes
 - verbs usually map to operations or associations

WEEK 3 : LECTURE 3 - CLASS MODELS & DIAGRAMS

Bachelor's Degree in Information Technology
BITS4111, SOFTWARE DESIGN

Trimester: 06

From March 2024 to June 2024

FINDING CLASSES AND ASSOCIATIONS Using Natural Language Analysis

Part of speech	model component	Examples
Proper noun	Instance (object)	Alice, Ace of Hearts
Common noun	Class (or attribute)	Field Officer, PlayingCard, value
Doing verb	Operation	Creates, submits, shuffles
Being verb	Inheritance	Is a kind of, is one of either
Having verb	Aggregation/Composition	Has, consists of, includes
Modal verb	Constraint	Must be
Adjective	Helps identify an attribute	a <i>yellow</i> ball (i.e. color)

FINDING CLASSES AND ASSOCIATIONS

Using Natural Language Analysis

Identifying different object types

- Finding Entity Objects
 - Some things to look for. These may be candidates for objects, or they may help identify objects:
 - Terms that are domain-specific in use cases
 - Recurring nouns
 - Real-world entities and activities tracked by system
 - Use good naming conventions.

FINDING CLASSES AND ASSOCIATIONS

Using Natural Language Analysis

Identifying different object types

- Finding Entity Objects
 - Example: In a ReportEmergency use case -- "A field officer submits information to the system by filling out a form and pressing the 'Send Report' button"
 - FieldOfficer is a real world entity that interacts with the system
 - This is also likely an actor from the use case
 - As an actor, FieldOfficer is an external entity

FINDING CLASSES AND ASSOCIATIONS

Using Natural Language Analysis

Identifying different object types

- Finding Entity Objects
 - Example: In a ReportEmergency use case -- "A field officer submits information to the system by filling out a form and pressing the 'Send Report' button"
 - But we see that the field officer submits information -- here's data to be tracked
 - We'll create the entity object type EmergencyReport, as that's the more common name for the information the officer submits

FINDING CLASSES AND ASSOCIATIONS

Using Natural Language Analysis

Identifying different object types

- Finding Boundary Objects
 - Identify general user interface controls that initiate a use case
 - Note: Don't bother with the visual details here.
This will evolve later
 - Identify forms or windows for entering data into a system
 - Identify messages used by system to respond to a user

FINDING CLASSES AND ASSOCIATIONS

Using Natural Language Analysis

Identifying different object types

- Finding Control Objects
 - Control objects can help manage communication and interaction of other objects
 - If a use case is complex and involves many objects, create a control object to manage the use case
 - Identify 1 control object per actor in a use case
 - Life span of control object should last through the use case

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

- A simple object-oriented analysis technique that includes the users and developers in the analysis process
- A CRC card is an index card with three parts:
 - Class -- name goes at the top of the card
 - Responsibilities -- as a list on the left side of the card
 - Collaborators -- as a list on the right side of the card

ClassName	
Responsibility	Collaborator

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

- Class
 - Represents a type of object being modeled
 - One card per class
- Responsibility
 - Something that the class knows (keeps track of) or does
 - These should be the high-level responsibilities. Not trying to list out all member functions here

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

- Responsibility
 - Example: class Mailbox in a voice mail system might have these responsibilities:
 - keep new and saved messages
 - manage the recorded greeting
- Collaborator
 - Another class that the current class has to work with to complete its responsibilities

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

- Collaborator
 - Could be a class that has information we need
 - Could be a class that helps perform a task
 - Typically, we list a class as a collaborator if we (the current class) need to call upon it to help complete our own responsibilities
 - Example: To successfully keep new and saved messages, the Mailbox class has to send them to a MessageQueue to be added and stored. So on the Mailbox card, we list MessageQueue as a collaborator

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

A CRC Cards Session

- CRC cards can be used as a brainstorming technique, for the purpose of:
 - Identifying objects/classes
 - Identifying what each object's purpose is (responsibilities)
 - Discovering the dependencies and relationships between objects (collaborators)

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

A CRC Cards Session

- A CRC card "session" involves users and developers:
 - Domain experts/users -- intended users of the system, people who know the business being modeled. Good to have a few of these
 - Developer/Analyst -- should have a couple members of the development team. People who understand OOP modeling and development processes
 - Facilitator -- one person who keeps things on track and progressing forward

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

A CRC Cards Session

- The process is based on going through use cases (or specific scenarios built from use cases), and using these to discover objects, responsibilities, and collaborators
- The general process:
 - Start with a scenario (usually representing a normal course through a use case)
 - Identify initial classes/objects and make cards for them (this is can often be done by picking out the nouns)

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

A CRC Cards Session

- The general process:
 - Going through a scenario helps identify responsibilities of a chosen object
 - Identify collaborations between objects that have been created
 - Sometimes, we'll identify a collaboration with a new object type that doesn't have a card yet -- this helps discover new classes

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

A CRC Cards Session

- The general process:
 - When new classes are created, walk through scenarios again to discover any new responsibilities and collaborators (it's an iterative process)
 - More use cases/scenarios will yield more classes, responsibilities, and collaborators

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

A CRC Cards Session

- Finding responsibilities
 - Look for verbs in the scenario descriptions. These often tell us what an object does
 - Also ask what the class knows. This tells us what an object needs to store. Sometimes a primary responsibility of a class is management of certain unique information

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

A CRC Cards Session

- Finding collaborators
 - If a class has a responsibility that required it to get, or modify, information it doesn't have on its own, it will need to collaborate with another class
 - Most often, one class specifically initiates the collaboration
 - Usually, the collaboration is a request for information or a request to do something

FINDING CLASSES AND ASSOCIATIONS

CRC Cards

A CRC Cards Session

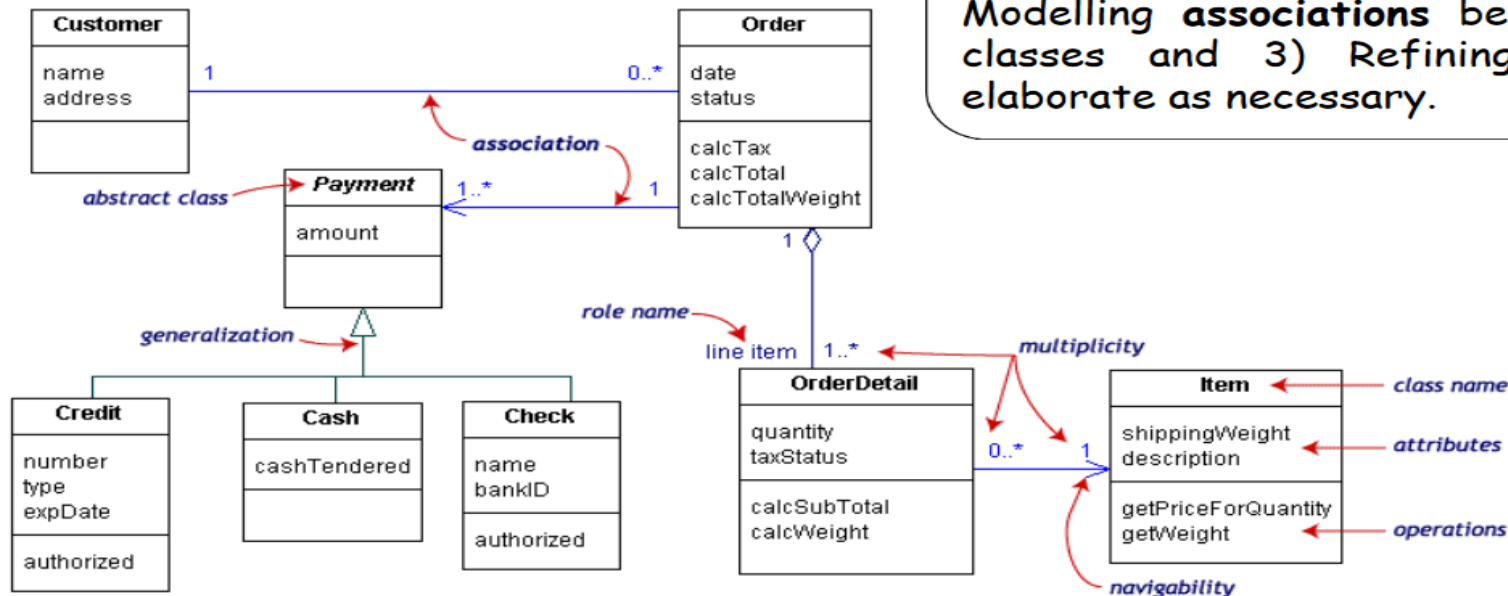
- Finding collaborators
 - The initiator's card should list the helper class as a collaborator
 - In this case, the initiator class depends on the collaborator class to accomplish its tasks

BASICS OF UML CLASS DIAGRAMS

- Once classes have been identified, they need to be modelled using class diagrams
- Class diagrams provide a structural view of systems.
- Class diagrams capture the static structure of Object-Oriented systems, or how they are structured rather than how they behave.
- Class diagrams support architectural design.
- Class diagrams represents the basics of Object- Oriented systems.
- They identify what classes there are, how they interrelate and how they interact.

BASICS OF UML CLASS DIAGRAMS

- Link to Requirements: Class diagrams constrain interactions & collaborations that support functional requirements.



These diagrams contain classes and associations. Construction involves: 1) Modelling **Classes**, 2) Modelling **associations** between classes and 3) Refining and elaborate as necessary.

BASICS OF UML CLASS DIAGRAMS

- They can be used throughout the development life cycle
- Class diagram carry different information depending on the phase of the development process and the level of detail being considered.
 - Initially, class diagrams reflect the problem domain, which is familiar to end-users
 - As development progresses, class diagrams move towards the implementation domain, which is familiar to software engineers
 - The contents of a class diagram will reflect this change in emphasis during the development process.

BASICS OF UML CLASS DIAGRAMS

Class Diagrams Rationale

- Desirable to build systems quickly and cheaply (and to meet requirements)
- Desirable to make the system easy to maintain and modify

BASICS OF UML CLASS DIAGRAMS

Class

- A class is a description of a group of objects all with similar roles in the system
 - Structural features define what objects of the class know
 - Behavioral features define what objects of the class can do

BASICS OF UML CLASS DIAGRAMS

Class

- Objects derive from:
 - Things: tangible, real-world objects, etc.
 - Roles: classes of actors in systems, e.g., students, managers, nurses, etc.
 - Events: admission, registration, matriculation, etc.
 - Interactions: meetings, tutorials, etc

BASICS OF UML CLASS DIAGRAMS

Class

Basic Class Compartments

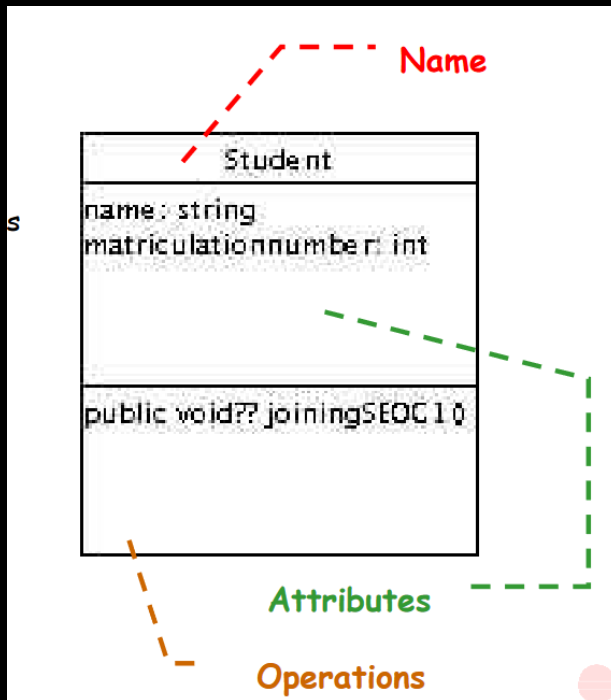
- Name
- Attributes
 - represent the state of an object of the class
 - Are descriptions of the structural or static features of a class
- Operations
 - define the way in which objects may interact
 - Operations are descriptions of behavioral or dynamic features of a class

BASICS OF UML CLASS DIAGRAMS

Class

Basic Class Compartments

- Objects are instances of classes



BASICS OF UML CLASS DIAGRAMS

Class

Attributes and Operations

- Syntax: < featureName >:<type>
- type is the data type of the attribute or the data
- returned by the operation
- Visibility: private (-), public (+) or protected (#)

BASICS OF UML CLASS DIAGRAMS

Class

Attributes

- Initial value, Derived Attribute, Multiplicity [m..n]
- Multiplicity: how many instances of one class can be associated with another class
- Examples of Multiplicity:
 - n..m - n to m instances;
 - 0..1 - zero or one instance;
 - 0..* or * - no limit on the number of instances
 - 1 - exactly one instance;
 - 1..* at least one instance

THE ATM CASE STUDY

- A requirements document specifies the overall purpose of a system and what it must do.
- A small local bank intends to install a new ATM
- The ATM should have the following functionality.
 - the screen prompts the user to enter an account number.
 - the user enters a five-digit account number, using the keypad.
 - for authentication purposes, the screen prompts the user to enter the PIN.

THE ATM CASE STUDY

- The ATM should have the following functionality.
 - The user enters a five-digit PIN, using the keypad.
 - If the user enters a valid account number and the correct PIN for that account, the screen displays the main menu.

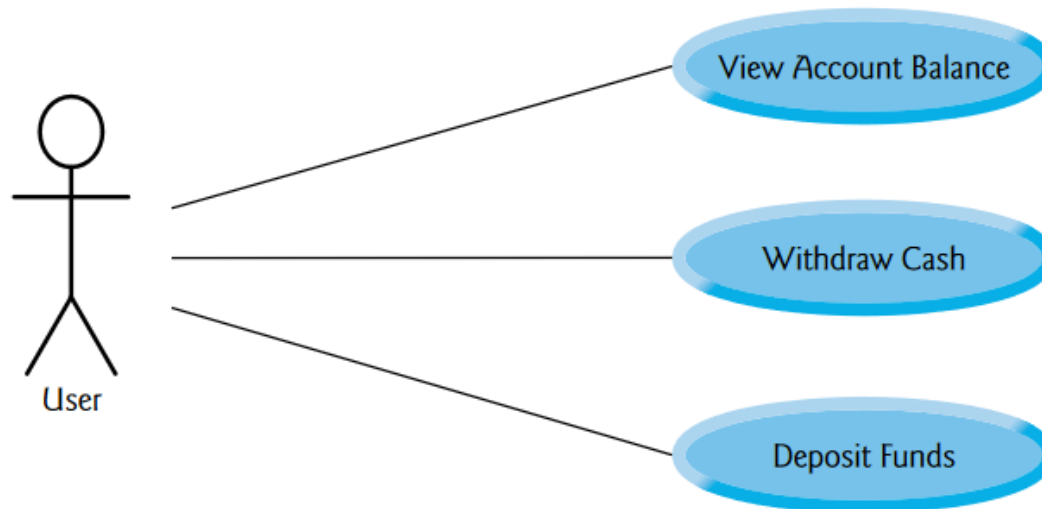
THE ATM CASE STUDY

- To make a deposit:
 - 1. The user chooses the deposit option
 - 2. The screen prompts the user to enter a deposit amount.
 - 3. The screen displays a message telling the user to insert a deposit envelope into the deposit slot.
 - 4. If the deposit slot receives the deposit envelope, the ATM credits the deposit amount to the user's account balance in the bank's database.

THE ATM CASE STUDY

- To capture what a proposed system should do, developers often employ a technique known as use case modeling.
- Use cases represent different capabilities that the system provides to its clients.
- The simplified ATM system we build in this case study requires only the first three use cases
- Use Case Diagrams
 - A use case diagram models the interactions between a system's clients and the system.
 - The next figure shows the use case diagram for our ATM system.

THE ATM CASE STUDY



THE ATM CASE STUDY

Identifying the Classes in the ATM Requirements Document

- We create classes only for the nouns and noun phrases in the ATM system as shown below.

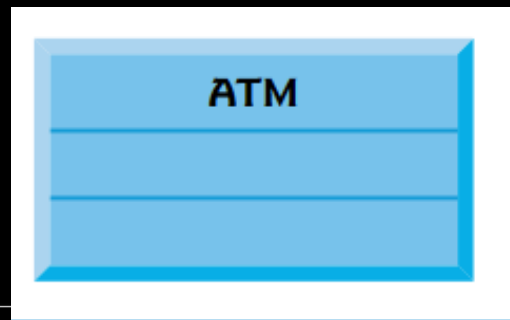
Nouns and noun phrases in the requirements document			
bank	money / fund	account number	ATM
screen	PIN	user	keypad
bank database	customer	cash dispenser	balance inquiry
transaction	\$20 bill / cash	withdrawal	account
deposit slot	deposit	balance	deposit envelope

- We do not need to model some nouns such as “bank” which are not part of the ATM operations.

THE ATM CASE STUDY

Identifying the Classes in the ATM Requirements Document

- UML class diagrams model the classes in the ATM system and their interrelationships
 - The top compartment contains the name of the class.
 - The middle compartment contains the class's attributes.
 - The bottom compartment contains the class's operations.



THE END